



PB CS2016

PROJECT PROPOSAL

FAST NUCES | BS FinTech – Semester II

Project Title	Finance Tracker
Course	CS2016 – Programming for Business
Program	BS Financial Technologies – Semester II
Instructor	Ali Asad Shah
Submission Date	April 21th, 2026

Group Members:

#	Student Name	Student ID	Role
1	Deepak Kumar	25K-5636	Lead Developer – Core Modules
2	Asma Atif	25K-5550	File Handling & Data Validation
3	Muhammad Razi Hussain	25K-5666	Testing & Documentation

DEPARTMENT OF FINTECH

FAST NATIONAL UNIVERSITY OF COMPUTER & EMERGING
SCIENCE KARACHI CAMPUS

Contents

1. Project Idea (Abstract)	3
2. Detailed Features	3
2.1 Core Features	3
2.2 Additional / Bonus Features	3
3. Methodology	3
3.1 Development Approach	3
3.2 Technology Stack	4
3.3 Division of Work	4
4. Expected Outcomes	4
5. Project Timeline	4
6. Potential Challenges	5
7. System Flowchart	5

1. Project Idea (Abstract)

PyFinance is a Python-based command-line personal finance management system designed to help individuals take control of their financial lives. In today's fast-paced world, many people struggle to track spending, manage debts, set savings goals, and stay within budgets — often due to the lack of an accessible, centralized tool. PyFinance addresses this gap by offering a multi-user platform where each registered user can securely log in and manage their complete financial picture from a single application.

The system allows users to record and categorize daily expenses, set category-based spending budgets with real-time warnings when limits are exceeded, track outstanding debts and log repayments, and maintain personal savings goals with progress monitoring. All data is persisted locally in CSV files, ensuring simplicity and portability. The application also generates visual reports using Matplotlib, including bar charts, line graphs, and comparative budget graphs, giving users an intuitive snapshot of their financial health. PyFinance targets students, young professionals, and anyone seeking a lightweight yet comprehensive financial tracker built entirely in Python.

2. Detailed Features

2.1 Core Features

- User Registration & Login — Secure multi-user system with username/password authentication stored in a CSV file. Each user gets an isolated financial profile.
- Expense Tracking — Users can add expenses with an amount, category (e.g., Food, Transport, Utilities), and date. All entries are validated and saved persistently.
- Debt Management — Add named debts with total amounts, view all outstanding debts, and log partial or full repayments with real-time remaining balance updates.
- Budget Management — Set spending limits per category. The system calculates total expenditure against each limit and warns the user when a budget is breached.
- Savings Goals — Users can set a savings target and incrementally log savings contributions, tracking progress toward their goal at any time.
- CSV File Handling — All data is stored and retrieved from structured CSV files using Python's built-in csv module.

2.2 Additional / Bonus Features

- Category-wise Expense Chart — Bar chart visualizing total spending broken down by category using Matplotlib.
- Monthly Expense Trend Graph — Line graph showing month-by-month expense totals, helping identify spending patterns over time.
- Savings Progress Chart — Comparative bar chart of the user's savings goal versus current savings amount.
- Debt Remaining Chart — Bar chart displaying remaining balances across all active debts.
- Budget Warning Graph — Side-by-side bar chart comparing budget limits to actual spending per category, with red highlights for exceeded limits.

3. Methodology

3.1 Development Approach

We will follow a modular, iterative development approach. The application is divided into independent functional modules (users, expenses, debts, budgets, savings, graphs), each

developed and tested separately before integration. Team members will work on their assigned modules in parallel, with regular sync-ups to ensure compatibility.

3.2 Technology Stack

- Language: Python 3.x
- Libraries: csv (file I/O), datetime (date validation), matplotlib (data visualization), os (file management)
- IDE: VS Code / PyCharm
- Version Control & Collaboration: GitHub
- Data Storage: Local CSV files (users.csv, expenses.csv, debts.csv, budgets.csv, savings.csv)

3.3 Division of Work

- **Deepak Kumar** — User authentication module, expense tracking module, main menu loop, application integration and all five Matplotlib graph functions
- **Asma Atif** — Debt management module, budget module, CSV read/write utilities, input validation across all modules
- **Razi Hussain**— Savings module, testing, debugging, and final documentation

4. Expected Outcomes

- A fully functional Python command-line application that enables multi-user personal finance management, covering expenses, debts, budgets, and savings.
- Persistent data storage across all five CSV files, ensuring user data is retained between sessions.
- Five interactive Matplotlib visualizations providing clear graphical insight into the user's financial status.
- Robust input validation and error handling throughout the application to prevent crashes and data corruption.
- Well-structured, commented source code organized into logical modules with a clear separation of concerns.
- A project report documenting system design, implementation decisions, testing results, and potential future improvements.

5. Project Timeline

Phase	Tasks
Phase 1	Requirements gathering, feature scoping, research on CSV handling and Matplotlib
Phase 2	System design: module architecture, CSV schema design, menu flow planning
Phase 3	Core development: user auth, expenses, debts, budgets, savings modules
Phase 4	Graph module development, full integration, testing, and bug fixing

6. Potential Challenges

- **Data Integrity Across CSV Files** — Ensuring referential consistency (e.g., `user_id` matching across files) requires careful validation. We plan to enforce ID checks on every read/write operation.
- **Input Validation & Edge Cases** — Handling invalid inputs such as non-numeric amounts, incorrect date formats, and empty strings without crashing. We will wrap all user inputs in `try/except` blocks with descriptive error messages.
- **Concurrent File Access** — The application reads and rewrites entire CSV files on each operation. We will use atomic write patterns to prevent data loss if the application exits mid-write.
- **Matplotlib Rendering** — Charts may not display correctly in all environments. We will test graph output across different setups and provide a fallback text-based summary if `plt.show()` is unavailable.
- **Team Coordination** — Working across modules in parallel risks integration conflicts. We will use GitHub branches per feature and conduct weekly merge reviews to keep the codebase consistent.

Declaration

We, the undersigned, declare that this project proposal is our original work and has not been submitted elsewhere for academic credit.

Member 1 Signature:



Member 2 Signature:



Member 3 Signature:



7. System Flowchart

The following flowchart illustrates the complete execution flow of the PyFinance application, from startup through user authentication to the feature modules and exit.

